Dokumentasi Data Product: Early Warning Aktivitas & Akurasi Siswa

Kevin Sugito

1. Gambaran Umum

Andalan School adalah platform belajar mirip LMS: guru membuat soal/tugas, siswa mengerjakan, lalu sistem menyimpan jawaban dan aktivitas. Data aktivitas ini bisa diolah jadi data product yang berguna untuk sekolah, salah satunya fitur early warning (peringatan dini).

Early warning ini fokus ke dua hal yang gampang diukur dari data jawaban: (1) aktivitas pengerjaan (seberapa sering siswa mengerjakan), dan (2) akurasi jawaban (seberapa sering benar). Dari situ, sekolah bisa cepat tahu siswa mana yang perlu dipantau sebelum masalahnya makin besar.

Output utama MVP:

- Email laporan rutin ke admin sekolah berisi ringkasan HIGH/MED/LOW dan daftar siswa HIGH+MED.
- Lampiran file CSV (HIGH+MED) supaya gampang didownload/diolah.
- Penyimpanan data mentah di MinIO dalam format Parquet (biar cepat diproses ulang).

2. Konsep dan Alur Sistem

Stack yang dipakai:

- PostgreSQL (sumber data utama, diakses lewat VPN).
- MinIO (object storage, menyimpan Parquet sebagai data lake sederhana).
- DuckDB (query/analitik lokal yang cepat untuk Parquet).
- Ollama (self-hosted LLM untuk membantu merapikan kalimat email).
- SMTP Gmail (mengirim email otomatis).

Alur datanya (ringkas):

1. Tarik data dari PostgreSQL (tabel jawaban/aktivitas) → simpan ke MinIO sebagai Parquet per hari (dt=YYYY-MM-DD).
2. DuckDB membaca Parquet dari MinIO → hitung fitur 14 hari terakhir dan 14 hari sebelumnya → tentukan level risiko.
3. Hasil analitik disimpan jadi CSV (risk_report.csv dan versi HIGH+MED).

4. Ollama membuat bagian pembuka dan saran tindakan (bahasa lebih enak dibaca).
5. Script mengirim email HTML (tabel rapi di Gmail) + attach CSV.

Kenapa bentuknya Parquet di MinIO?

Karena Parquet lebih hemat storage dan lebih cepat dibaca untuk analitik dibanding tarik langsung dari database tiap kali. Selain itu, untuk demo dan MVP, MinIO sudah cukup sebagai 'data lake' versi ringan.

3. Logika Risk (MVP)

Agar tidak kejebak weekend/libur, logika dibuat pakai perbandingan dua window yang sama panjang:

Istilah yang dipakai:

- attempts_14d: jumlah attempt/jawaban yang dibuat siswa dalam 14 hari terakhir (window sekarang).
- attempts_prev14d: jumlah attempt/jawaban dalam 14 hari sebelumnya (window pembanding).
- drop_pct: persentase penurunan aktivitas = (attempts_prev14d - attempts_14d) / attempts_prev14d.
- accuracy_14d: rata-rata benar (is_correct) dalam 14 hari terakhir (kalau datanya ada).
- days_since_last_activity: selisih hari dari aktivitas terakhir ke tanggal data terbaru (as_of).

Aturan level risiko bisa diubah lewat parameter di script (threshold). Contoh aturan yang dipakai di MVP:

- HIGH: aktivitas 14 hari terakhir = 0 padahal 14 hari sebelumnya cukup aktif (mis. >= 10), atau akurasi rendah dengan attempt cukup banyak, atau drop aktivitas sangat besar.
- MED: aktivitas turun cukup besar dibanding 14 hari sebelumnya (mis. drop >= 60%), atau attempt 14 hari terakhir sangat kecil.
- LOW: selain itu (normal).

Catatan: Karena ini MVP, fokusnya bukan menentukan 'naik kelas atau tidak', tapi memberi sinyal awal supaya sekolah bisa follow-up dulu.

4. Setup dan Konfigurasi

Prasyarat:

- Akses VPN ke server PostgreSQL.
- Python + venv (untuk menjalankan script).
- Docker Desktop (untuk MinIO, pgAdmin, dan Ollama).
- Bucket MinIO bernama 'andalan-lake'.

File konfigurasi yang dipakai:

- .env (menyimpan credential PG, MinIO, SMTP, dan nama pengirim).

5. Kode Program

1) Docker-Compose.yaml

```
services:
  pgadmin:
    image: dpage/pgadmin4:latest
    container_name: pgadmin
    restart: "no"
    ports:
      - "5050:80"
    environment:
      PGADMIN_DEFAULT_EMAIL: kevin@local.dev
      PGADMIN_DEFAULT_PASSWORD: admin123
    volumes:
      - pgadmin_data:/var/lib/pgadmin

  minio:
    image: minio/minio:latest
    container_name: minio
    restart: "no"
    ports:
      - "9000:9000"
      - "9001:9001"
    environment:
      MINIO_ROOT_USER: admin123
      MINIO_ROOT_PASSWORD: admin123
    command: server /data --console-address ":9001"
    volumes:
      - minio_data:/data

  ollama:
    image: ollama/ollama:latest
```

```
    container_name: ollama
    restart: "no"
    ports:
      - "11434:11434"
    volumes:
      - ollama_data:/root/.ollama

volumes:
  pgadmin_data:
  minio_data:
  ollama_data:
```

Penjelasan singkat:

Docker Compose menyalakan tiga service: pgAdmin (opsional untuk cek data), MinIO (storage Parquet), dan Ollama (LLM).

2) **Script 1 - Extract PG ke MinIO (Parquet)**

Nama file: 01_full_load_pg_to_minio_parquet.py

```
import os
from datetime import datetime, timedelta, date
import pandas as pd
from sqlalchemy import create_engine, text
from dotenv import load_dotenv

import pyarrow as pa
import pyarrow.parquet as pq
import s3fs

load_dotenv()

TABLE_SCHEMA = "public"
TABLE_NAME = "kp_ptn_practice_user_answer"

DATE_COL = "created_at"
DELETED_COL = "deleted_at"

S3_PREFIX = f"bronze/{TABLE_SCHEMA}.{TABLE_NAME}"

CHUNK_SIZE = 20000
```

```
PURGE_PREFIX_BEFORE_LOAD = False

DELETE_OBJECT_BEFORE_WRITE = True

def get_pg_engine():
    pg_host = os.environ["PG_HOST"]
    pg_port = os.environ.get("PG_PORT", "5432")
    pg_db = os.environ["PG_DB"]
    pg_user = os.environ["PG_USER"]
    pg_pwd = os.environ["PG_PASSWORD"]
    url = f"postgresql+psycopg2://{pg_user}:{pg_pwd}@{pg_host}:{pg_port}/{pg_db}"
    return create_engine(url)

def get_s3_fs():
    endpoint = os.environ["MINIO_ENDPOINT"].rstrip("/")
    key = os.environ["MINIO_ACCESS_KEY"]
    secret = os.environ["MINIO_SECRET_KEY"]
    return s3fs.S3FileSystem(
        key=key,
        secret=secret,
        client_kwargs={"endpoint_url": endpoint, "region_name": "us-east-1"},
    )

def s3_key(bucket: str, key: str) -> str:
    return f"{bucket}/{key}"

def daterange(d1: date, d2: date):
    cur = d1
    while cur <= d2:
        yield cur
        cur += timedelta(days=1)

def purge_prefix(fs: s3fs.S3FileSystem, bucket: str, prefix: str):
    full_prefix = f"{bucket}/{prefix}"
    if not fs.exists(full_prefix):
        return
    paths = fs.find(full_prefix)
    if paths:
        fs.rm(paths, recursive=True)

def main():
    bucket = os.environ["MINIO_BUCKET"]
    engine = get_pg_engine()
    fs = get_s3_fs()
```

```
    try:
        fs.ls("")
    except Exception as e:
        raise RuntimeError(f"MinIO auth/endpoint failed: {e}") from e

    if not fs.exists(bucket):
        raise RuntimeError(f"Bucket '{bucket}' tidak ditemukan. Buat dulu di MinIO
Console (localhost:9001).")

    full_table = f"{TABLE_SCHEMA}.{TABLE_NAME}"

    where_valid = ""
    if DELETED_COL:
        where_valid = f"WHERE {DELETED_COL} IS NULL"

    q_minmax = f"""
        SELECT MIN({DATE_COL}) AS min_dt, MAX({DATE_COL}) AS max_dt,
COUNT(*) AS total_rows
        FROM {full_table}
        {where_valid};
    """
    with engine.connect() as conn:
        row = conn.execute(text(q_minmax)).mappings().one()

    min_dt = row["min_dt"]
    max_dt = row["max_dt"]
    total_rows = row["total_rows"]

    if not max_dt or not min_dt:
        raise RuntimeError("MIN/MAX created_at kosong. Cek tabel & kolom created_at.")

    start_date = min_dt.date()
    end_date = max_dt.date()

    print(f"[INFO] {full_table} total_rows={total_rows}")
    print(f"[INFO] FULL LOAD range={start_date} .. {end_date}")

    if PURGE_PREFIX_BEFORE_LOAD:
        print(f"[WARN] PURGE enabled. Removing s3://{bucket}/{S3_PREFIX}/ ...")
        purge_prefix(fs, bucket, S3_PREFIX)
        print("[OK] Purge done.")

    for d in daterange(start_date, end_date):
        day_start = datetime(d.year, d.month, d.day, 0, 0, 0)
        day_end = day_start + timedelta(days=1)
```

```
        filters = [f"{DATE_COL} >= :day_start", f"{DATE_COL} < :day_end"]
        if DELETED_COL:
            filters.append(f"{DELETED_COL} IS NULL")
        where_clause = "WHERE " + " AND ".join(filters)

        q_count = f"SELECT COUNT(*) FROM {full_table} {where_clause};"
        with engine.connect() as conn:
            c = conn.execute(text(q_count), {"day_start": day_start, "day_end":
day_end}).scalar()

        if c == 0:
            continue

        key_prefix = f"{S3_PREFIX}/dt={d.isoformat()}"
        out_key = f"{key_prefix}/part-000.parquet"
        out_path = s3_key(bucket, out_key)

        print(f"[INFO] dt={d} rows={c} -> writing {out_path}")

        q_data = f"SELECT * FROM {full_table} {where_clause} ORDER BY
{DATE_COL} ASC;"
        chunks = []
        for df in pd.read_sql_query(
            sql=text(q_data),
            con=engine,
            params={"day_start": day_start, "day_end": day_end},
            chunksize=CHUNK_SIZE,
        ):
            chunks.append(df)

        df_all = pd.concat(chunks, ignore_index=True)
        table = pa.Table.from_pandas(df_all, preserve_index=False)

        if DELETE_OBJECT_BEFORE_WRITE and fs.exists(out_path):
            fs.rm(out_path)

        with fs.open(out_path, "wb") as f_out:
            pq.write_table(table, f_out, compression="snappy")

        print(f"[OK] wrote s3://{out_path}")

    print("[DONE] FULL LOAD finished.")

if __name__ == "__main__":
    main()
```

Fungsi script 1: mengambil semua data dari tabel target di PostgreSQL, lalu menyimpannya ke MinIO sebagai Parquet per hari (partition dt=...).

3) **Script 2 - Hitung Risk dengan DuckDB**

Nama file: 02_compute_risk_duckdb_from_minio.py

```
import os
import duckdb
import pandas as pd
from dotenv import load_dotenv

load_dotenv()

MINIO_BUCKET = os.environ["MINIO_BUCKET"]
MINIO_ENDPOINT = os.environ["MINIO_ENDPOINT"]
MINIO_ACCESS_KEY = os.environ["MINIO_ACCESS_KEY"]
MINIO_SECRET_KEY = os.environ["MINIO_SECRET_KEY"]

PARQUET_GLOB =
f"s3://{MINIO_BUCKET}/bronze/public.kp_ptn_practice_user_answer/dt=*/part-
*.parquet"

WINDOW_DAYS = 14

MIN_ATTEMPTS_FOR_ACCURACY = 15
LOW_ACCURACY_THRESHOLD = 0.40
DROP_PCT_THRESHOLD_MED = 0.60
DROP_PCT_THRESHOLD_HIGH = 0.80

OUT_DIR = "out"
OUT_CSV = os.path.join(OUT_DIR, "risk_report.csv")

def main():
    os.makedirs(OUT_DIR, exist_ok=True)

    con = duckdb.connect()

    con.execute("INSTALL httpfs;")
    con.execute("LOAD httpfs;")

    endpoint_host = MINIO_ENDPOINT.replace("http://", "").replace("https://",
"").rstrip("/")

    con.execute(f"SET s3_endpoint='{endpoint_host}';")
```

```
    con.execute(f"SET s3_access_key_id='{MINIO_ACCESS_KEY}';")
    con.execute(f"SET s3_secret_access_key='{MINIO_SECRET_KEY}';")
    con.execute("SET s3_use_ssl=false;")
    con.execute("SET s3_url_style='path';")

    con.execute(f"""
        CREATE OR REPLACE VIEW answers AS
        SELECT
            practice_user_history_id AS student_key,
            created_at::TIMESTAMP AS created_at,
            is_correct,
            deleted_at
        FROM read_parquet('{PARQUET_GLOB}', union_by_name=true);
    """)

    as_of = con.execute("""
        SELECT MAX(created_at) AS as_of
        FROM answers
        WHERE deleted_at IS NULL
    """).fetchone()[0]

    if as_of is None:
        raise RuntimeError("Tidak ada data (as_of NULL). Cek parquet path / bucket.")

    print(f"[INFO] as_of (max created_at) = {as_of}")

    con.execute(f"""
        CREATE OR REPLACE TEMP VIEW base AS
        SELECT
            student_key,
            created_at,
            is_correct
        FROM answers
        WHERE deleted_at IS NULL
          AND created_at >= TIMESTAMP '{as_of}' - INTERVAL '{WINDOW_DAYS*2}
days'
          AND created_at <= TIMESTAMP '{as_of}';
    """)

    con.execute(f"""
        CREATE OR REPLACE TEMP VIEW feat AS
        SELECT
            student_key,

            COUNT(*) FILTER (
                WHERE created_at >  TIMESTAMP '{as_of}' - INTERVAL
```

```
'{WINDOW_DAYS} days'
            AND created_at <= TIMESTAMP '{as_of}'
        ) AS attempts_14d,

        COUNT(DISTINCT DATE(created_at)) FILTER (
          WHERE created_at >  TIMESTAMP '{as_of}' - INTERVAL
'{WINDOW_DAYS} days'
            AND created_at <= TIMESTAMP '{as_of}'
        ) AS active_days_14d,

        AVG(CASE
          WHEN created_at >  TIMESTAMP '{as_of}' - INTERVAL
'{WINDOW_DAYS} days'
           AND created_at <= TIMESTAMP '{as_of}'
           AND is_correct IS NOT NULL
          THEN CASE WHEN is_correct THEN 1.0 ELSE 0.0 END
          ELSE NULL
        END) AS accuracy_14d,

        COUNT(*) FILTER (
          WHERE created_at >  TIMESTAMP '{as_of}' - INTERVAL
'{WINDOW_DAYS*2} days'
            AND created_at <= TIMESTAMP '{as_of}' - INTERVAL
'{WINDOW_DAYS} days'
        ) AS attempts_prev14d,

        AVG(CASE
          WHEN created_at >  TIMESTAMP '{as_of}' - INTERVAL
'{WINDOW_DAYS*2} days'
           AND created_at <= TIMESTAMP '{as_of}' - INTERVAL
'{WINDOW_DAYS} days'
           AND is_correct IS NOT NULL
          THEN CASE WHEN is_correct THEN 1.0 ELSE 0.0 END
          ELSE NULL
        END) AS accuracy_prev14d,

        MAX(created_at) AS last_activity
      FROM base
      GROUP BY student_key;
  """)

  con.execute(f"""
      CREATE OR REPLACE TEMP VIEW risk AS
      SELECT
        student_key,
        attempts_14d,
```

```
        active_days_14d,
        accuracy_14d,
        attempts_prev14d,
        accuracy_prev14d,
        last_activity,

        CASE
          WHEN attempts_prev14d > 0 THEN
            (attempts_prev14d - attempts_14d) * 1.0 / attempts_prev14d
          ELSE NULL
        END AS drop_pct,

        DATE_DIFF('day', DATE(last_activity), DATE(TIMESTAMP '{as_of}')) AS
days_since_last_activity,

        CASE
          WHEN
            (attempts_14d = 0 AND attempts_prev14d >= 10)
            OR
            (attempts_14d >= {MIN_ATTEMPTS_FOR_ACCURACY} AND
accuracy_14d IS NOT NULL AND accuracy_14d <
{LOW_ACCURACY_THRESHOLD})
            OR
            (attempts_prev14d >= 10 AND drop_pct IS NOT NULL AND drop_pct >=
{DROP_PCT_THRESHOLD_HIGH})
          THEN 'HIGH'

          WHEN
            (attempts_14d <= 3 AND attempts_prev14d >= 5)
            OR
            (attempts_prev14d >= 5 AND drop_pct IS NOT NULL AND drop_pct >=
{DROP_PCT_THRESHOLD_MED})
            OR
            (attempts_14d >= {MIN_ATTEMPTS_FOR_ACCURACY} AND
accuracy_14d IS NOT NULL AND accuracy_14d >=
{LOW_ACCURACY_THRESHOLD} AND accuracy_14d < 0.55)
          THEN 'MED'

          ELSE 'LOW'
        END AS risk_level,

        TRIM(BOTH ';' FROM
          CONCAT(
            CASE WHEN attempts_14d = 0 AND attempts_prev14d >= 10 THEN 'No
activity in last 14d but active in previous 14d; ' ELSE '' END,
            CASE WHEN attempts_prev14d >= 10 AND drop_pct IS NOT NULL AND
```

```
drop_pct >= {DROP_PCT_THRESHOLD_HIGH} THEN 'Activity dropped >= 80% vs
previous 14d; ' ELSE '' END,
            CASE WHEN attempts_prev14d >= 5  AND drop_pct IS NOT NULL AND
drop_pct >= {DROP_PCT_THRESHOLD_MED} THEN 'Activity dropped >= 60% vs
previous 14d; ' ELSE '' END,
            CASE WHEN attempts_14d >= {MIN_ATTEMPTS_FOR_ACCURACY}
AND accuracy_14d IS NOT NULL AND accuracy_14d <
{LOW_ACCURACY_THRESHOLD} THEN 'Low accuracy (<40%) with sufficient
attempts; ' ELSE '' END,
            CASE WHEN attempts_14d >= {MIN_ATTEMPTS_FOR_ACCURACY}
AND accuracy_14d IS NOT NULL AND accuracy_14d >=
{LOW_ACCURACY_THRESHOLD} AND accuracy_14d < 0.55 THEN 'Moderate
accuracy (40–55%) with sufficient attempts; ' ELSE '' END,
            CASE WHEN attempts_14d <= 3 AND attempts_prev14d >= 5 THEN 'Very
low attempts in last 14d; ' ELSE '' END
          )
        ) AS reason
      FROM feat;
  """)

  df = con.execute("""
      SELECT *
      FROM risk
      ORDER BY
        CASE risk_level WHEN 'HIGH' THEN 1 WHEN 'MED' THEN 2 ELSE 3 END,
        attempts_prev14d DESC,
        attempts_14d ASC
  """).df()

  df.to_csv(OUT_CSV, index=False)
  print(f"[OK] risk report saved: {OUT_CSV}")

  print("\n=== TOP 20 RISK STUDENTS ===")
  print(df.head(20).to_string(index=False))

if __name__ == "__main__":
  main()
```

Fungsi script 2: baca Parquet dari MinIO, hitung metric 14 hari terakhir vs 14 hari sebelumnya, lalu output risk_report.csv.

4) **Script 3 - Email HTML + Attach CSV (pakai Ollama)**

Nama file: 03_generate_and_send_email.py

```
import os
import ssl
import smtplib
import pandas as pd
import requests
from dotenv import load_dotenv
from email.mime.text import MIMEText
from email.mime.multipart import MIMEMultipart
from email.mime.base import MIMEBase
from email import encoders

load_dotenv()

RISK_CSV = "out/risk_report.csv"

OLLAMA_URL = os.environ.get("OLLAMA_URL", "http://localhost:11434")
OLLAMA_MODEL = os.environ.get("OLLAMA_MODEL", "qwen2.5:3b")

SMTP_HOST = os.environ.get("SMTP_HOST", "smtp.gmail.com")
SMTP_PORT = int(os.environ.get("SMTP_PORT", "587"))
SMTP_USER = os.environ.get("SMTP_USER")
SMTP_APP_PASSWORD = os.environ.get("SMTP_APP_PASSWORD")

EMAIL_TO = os.environ.get("EMAIL_TO", SMTP_USER)
SENDER_NAME = os.environ.get("SENDER_NAME", "Kevin")

SEND_EMAIL = True
MAX_ROWS_IN_EMAIL = 200

def call_ollama(prompt: str) -> str:
    url = f"{OLLAMA_URL.rstrip('/')}/api/generate"
    payload = {
        "model": OLLAMA_MODEL,
        "prompt": prompt,
        "stream": False,
        "options": {"num_predict": 300},
    }
    r = requests.post(url, json=payload, timeout=180)
    r.raise_for_status()
    return (r.json().get("response") or "").strip()

def html_escape(s: str) -> str:
    return (
```

```
        str(s)
        .replace("&", "&amp;")
        .replace("<", "&lt;")
        .replace(">", "&gt;")
        .replace("", "&quot;")
    )

def build_table_html(df: pd.DataFrame, max_rows: int) -> str:
    cols = [
        "student_key",
        "risk_level",
        "attempts_14d",
        "accuracy_14d",
        "attempts_prev14d",
        "accuracy_prev14d",
        "drop_pct",
        "days_since_last_activity",
        "reason",
    ]
    cols = [c for c in cols if c in df.columns]
    d = df[cols].copy().head(max_rows)

    for c in ["accuracy_14d", "accuracy_prev14d", "drop_pct"]:
        if c in d.columns:
            d[c] = d[c].apply(lambda x: "" if pd.isna(x) else f"{float(x):.2f}")

    if "reason" in d.columns:
        d["reason"] = d["reason"].fillna("").astype(str).str.replace("\n", " ").str.strip()

    ths = "".join([f"<th style='border:1px solid #ddd;padding:8px;text-
align:left;background:#f7f7f7'>{html_escape(c)}</th>" for c in cols])

    rows_html = []
    for _, r in d.iterrows():
        tds = ""
        for c in cols:
            val = "" if pd.isna(r[c]) else r[c]
            tds += f"<td style='border:1px solid #ddd;padding:8px;vertical-
align:top'>{html_escape(val)}</td>"
        rows_html.append(f"<tr>{tds}</tr>")

    return f"""
    <table style="border-collapse:collapse;width:100%;font-size:13px">
      <thead><tr>{ths}</tr></thead>
      <tbody>
        {".join(rows_html)}
```

```
    </tbody>
  </table>
  """.strip()

def attach_file(msg: MIMEMultipart, filepath: str, filename: str):
    part = MIMEBase("application", "octet-stream")
    with open(filepath, "rb") as f:
        part.set_payload(f.read())
    encoders.encode_base64(part)
    part.add_header("Content-Disposition", f'attachment; filename="{filename}"')
    msg.attach(part)

def send_email_html(subject: str, html_body: str, to_addr: str, attachments: list[str] |
None = None):
    if not SMTP_USER or not SMTP_APP_PASSWORD:
        raise RuntimeError("SMTP_USER/SMTP_APP_PASSWORD belum diisi di .env")

    msg = MIMEMultipart()
    msg["From"] = SMTP_USER
    msg["To"] = to_addr
    msg["Subject"] = subject

    msg.attach(MIMEText(html_body, "html", "utf-8"))

    if attachments:
        for path in attachments:
            attach_file(msg, path, os.path.basename(path))

    context = ssl.create_default_context()
    with smtplib.SMTP(SMTP_HOST, SMTP_PORT) as server:
        server.ehlo()
        server.starttls(context=context)
        server.login(SMTP_USER, SMTP_APP_PASSWORD)
        server.sendmail(SMTP_USER, [to_addr], msg.as_string())

def main():
    if not os.path.exists(RISK_CSV):
        raise FileNotFoundError(f"{RISK_CSV} tidak ditemukan. Jalankan script 2 dulu.")

    df = pd.read_csv(RISK_CSV)

    counts = df["risk_level"].value_counts().to_dict()
    high_n = int(counts.get("HIGH", 0))
    med_n = int(counts.get("MED", 0))
    low_n = int(counts.get("LOW", 0))
```

```
    df_alert = df[df["risk_level"].isin(["HIGH", "MED"])].copy()

    order = {"HIGH": 0, "MED": 1, "LOW": 2}
    df_alert["_ord"] = df_alert["risk_level"].map(order).fillna(9)
    sort_cols = ["_ord"]
    if "attempts_prev14d" in df_alert.columns:
        sort_cols.append("attempts_prev14d")
    if "attempts_14d" in df_alert.columns:
        sort_cols.append("attempts_14d")
    df_alert = df_alert.sort_values(sort_cols, ascending=[True, False,
True]).drop(columns=["_ord"])

    os.makedirs("out", exist_ok=True)
    alert_csv_path = "out/risk_alert_high_med.csv"
    df_alert.to_csv(alert_csv_path, index=False)

    llm_prompt = """
Tulis 2 bagian email dalam Bahasa Indonesia yang natural dan profesional untuk admin
sekolah.
JANGAN menulis angka HIGH/MED/LOW, jangan menulis tabel, jangan menulis tanda
tangan.

Bagian 1 (Intro, 2-3 kalimat):
- Tujuan: early warning aktivitas & akurasi siswa di platform.

Bagian 2 (Tindakan yang disarankan, max 4 bullet):
- Fokus follow-up siswa risiko HIGH/MED.
- Hindari typo dan jangan campur bahasa Inggris.
""".strip()

    try:
        llm_text = call_ollama(llm_prompt)
    except Exception:
        llm_text = (
            "Halo Admin Sekolah,<br><br>"
            "Berikut kami sampaikan laporan early warning terkait aktivitas pengerjaan dan
akurasi jawaban siswa di platform Andalan School. "
            "Tujuannya agar sekolah bisa lebih cepat melakukan follow-up pada siswa yang
mulai menunjukkan penurunan tren.<br><br>"
            "<ul>"
            "<li>Prioritaskan follow-up siswa risiko HIGH terlebih dulu, lalu MED.</li>"
            "<li>Koordinasi dengan wali kelas untuk mengecek kendala (akses, motivasi,
pemahaman materi).</li>"
            "<li>Jika perlu, berikan pengingat/remedial singkat untuk topik yang paling
sering salah.</li>"
            "<li>Lakukan pengecekan ulang 7 hari ke depan untuk melihat apakah tren
```

```
membaik.</li>"
        "</ul>"
    )

  table_html = build_table_html(df_alert, MAX_ROWS_IN_EMAIL)

  subject = f"[Andalan School] Early Warning (HIGH={high_n}, MED={med_n})"

  html_body = f"""
  <div style="font-family:Arial, sans-serif; font-size:14px; line-height:1.5">
    {llm_text.replace("\n", "<br>")}

    <hr style="border:none;border-top:1px solid #eee;margin:16px 0" />

    <b>Ringkasan:</b>
    <ul>
      <li><b>HIGH:</b> {high_n} siswa</li>
      <li><b>MED:</b> {med_n} siswa</li>
      <li><b>LOW:</b> {low_n} siswa</li>
    </ul>

    <b>Definisi singkat:</b>
    <ul>
      <li><b>HIGH:</b> risiko tinggi, perlu follow-up cepat.</li>
      <li><b>MED:</b> perlu dipantau dan diingatkan.</li>
      <li><b>LOW:</b> normal (tidak perlu detail).</li>
    </ul>

    <b>Daftar siswa HIGH + MED</b> (student_key sebagai identitas
sementara):<br><br>
    {table_html}

    <br>
    <i>File CSV (HIGH+MED) terlampir untuk diunduh.</i>
    <br><br>

    Terima kasih.<br><br>
    Salam,<br>
    <b>{html_escape(SENDER_NAME)}</b>
  </div>
  """.strip()

  print("\n===== EMAIL PREVIEW (HTML) =====")
  print("To:", EMAIL_TO)
  print("Subject:", subject)
  print("(HTML body generated; check email inbox for rendering)")
```

```
    if not SEND_EMAIL:
        print("[INFO] SEND_EMAIL=False → email tidak dikirim.")
        return

    send_email_html(subject, html_body, EMAIL_TO, attachments=[alert_csv_path])
    print(f"[OK] Email sent to {EMAIL_TO} with attachment: {alert_csv_path}")

if __name__ == "__main__":
    main()
```

Fungsi script 3: ambil risk_report.csv, filter HIGH+MED, buat tabel HTML biar rapi di Gmail, lalu kirim email + lampiran CSV.

6. Uji Coba Singkat

Langkah 0 - Nyalakan service

Jalankan:

[RUN] docker compose up -d

Buat bucket MinIO (kalau belum):

Buka MinIO Console: http://localhost:9001 → buat bucket: andalan-lake

Tarik model Ollama yang ringan:

[RUN] docker exec -it ollama ollama pull qwen2.5:3b

Langkah 1 - Jalankan Script 1 (Extract)

[RUN] python 01_full_load_pg_to_minio_parquet.py

Hasil yang diharapkan: object Parquet muncul di MinIO pada path bronze/public.kp_ptn_practice_user_answer/dt=YYYY-MM-DD/part-000.parquet.

Langkah 2 - Jalankan Script 2 (Compute Risk)

[RUN] python 02_compute_risk_duckdb_from_minio.py

Hasil yang diharapkan: file out/risk_report.csv terbentuk. Kolom utama: student_key, risk_level, attempts_14d, attempts_prev14d, drop_pct, days_since_last_activity.

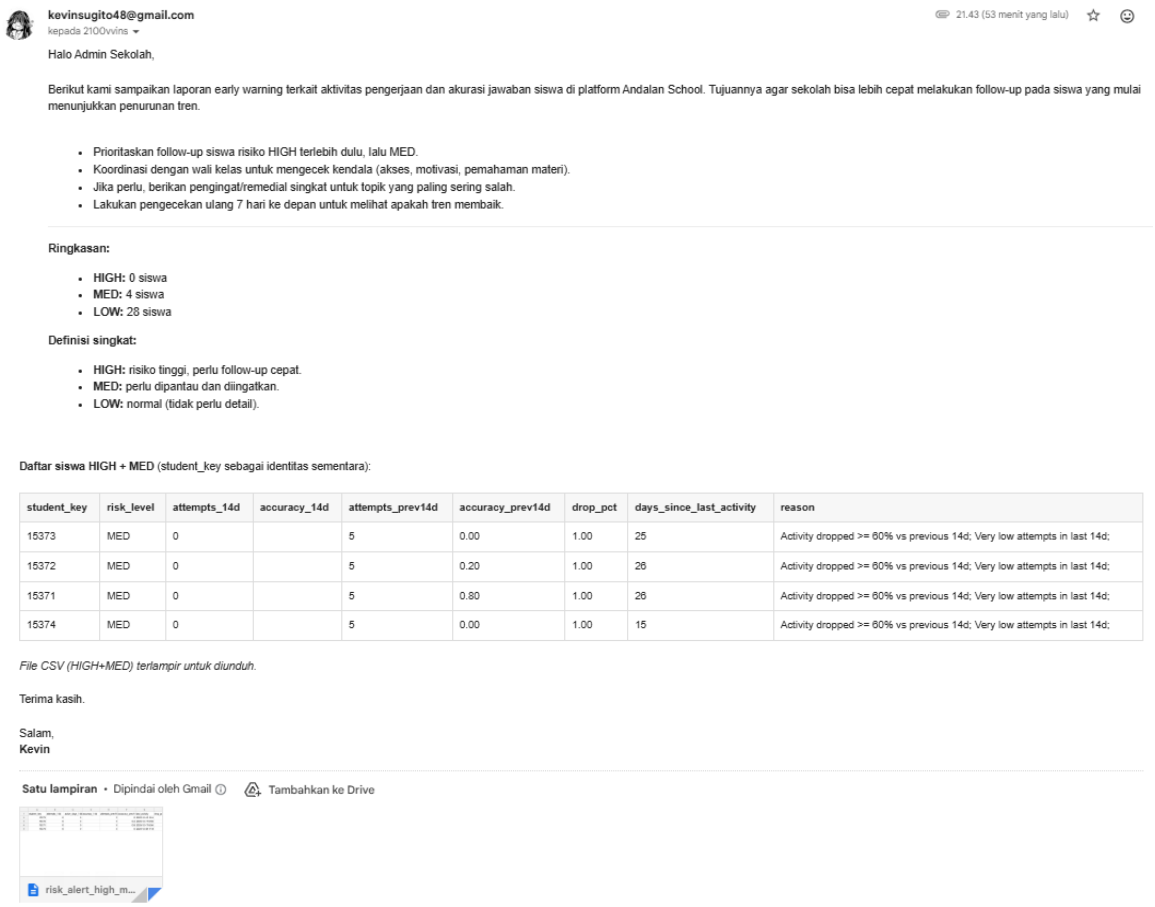
student_key	attempts_14d	active_days_14d	accuracy_14d	attempts_prev14d	accuracy_prev14d	last_activity	drop_pct	days_since_last_activity	risk_level	reason
15373	0	0		5	0.0	2025-12-18 19:42:43.753989	1.0	25	MED	Activity dropped >= 60% vs previous 14d
15372	0	0		5	0.2	2025-12-17 06:56:27.950602	1.0	26	MED	Activity dropped >= 60% vs previous 14d
15371	0	0		5	0.8	2025-12-17 06:54:35.723644	1.0	26	MED	Activity dropped >= 60% vs previous 14d
15374	0	0		5	0.0	2025-12-28 17:50:32.372055	1.0	15	MED	Activity dropped >= 60% vs previous 14d
15399	5	1	0.4	0		2026-01-11 19:27:01.490500		1	LOW	
15396	5	1	0.8	0		2026-01-11 19:09:59.875938		1	LOW	
15387	5	1	0.4	0		2026-01-09 19:15:15.518673		3	LOW	
15389	5	1	0.0	0		2026-01-11 17:07:18.118456		1	LOW	
15394	5	1	0.4	0		2026-01-11 18:47:02.654212		1	LOW	
15397	5	1	0.8	0		2026-01-11 19:17:42.747441		1	LOW	
15375	5	1	0.4	0		2026-01-05 18:45:37.156881		7	LOW	
15382	5	1	1.0	0		2026-01-09 17:20:40.621478		3	LOW	
15384	5	1	0.2	0		2026-01-09 18:56:18.039878		3	LOW	
15378	5	1	0.8	0		2026-01-08 19:05:09.461502		4	LOW	
15379	5	1	0.8	0		2026-01-08 19:17:21.847701		4	LOW	
15398	5	1	0.6	0		2026-01-11 19:23:37.648947		1	LOW	
15402	5	1	0.6	0		2026-01-12 19:31:44.838111		0	LOW	
15400	5	1	0.8	0		2026-01-11 19:37:21.169590		1	LOW	
15383	5	1	0.2	0		2026-01-09 18:47:11.329896		3	LOW	
15380	5	1	0.6	0		2026-01-08 19:28:13.333706		4	LOW	

risk_report

Langkah 3 - Jalankan Script 3 (Generate & Send Email)

[RUN] python 03_generate_and_send_email.py

Hasil yang diharapkan: email masuk ke inbox penerima dengan tabel HTML rapi dan ada lampiran CSV (HIGH+MED).



kevinsugito48@gmail.com
kepada 2100vvins

21.43 (53 menit yang lalu)

Halo Admin Sekolah,

Berikut kami sampaikan laporan early warning terkait aktivitas pengerjaan dan akurasi jawaban siswa di platform Andalan School. Tujuannya agar sekolah bisa lebih cepat melakukan follow-up pada siswa yang mulai menunjukkan penurunan tren.

- Prioritaskan follow-up siswa risiko HIGH terlebih dulu, lalu MED.
- Koordinasi dengan wali kelas untuk mengecek kendala (akses, motivasi, pemahaman materi).
- Jika perlu, berikan pengingat/remedial singkat untuk topik yang paling sering salah.
- Lakukan pengecekan ulang 7 hari ke depan untuk melihat apakah tren membaik.

Ringkasan:

- **HIGH:** 0 siswa
- **MED:** 4 siswa
- **LOW:** 28 siswa

Definisi singkat:

- **HIGH:** risiko tinggi, perlu follow-up cepat.
- **MED:** perlu dipantau dan diingatkan.
- **LOW:** normal (tidak perlu detail).

Daftar siswa HIGH + MED (student_key sebagai identitas sementara):

student_key	risk_level	attempts_14d	accuracy_14d	attempts_prev14d	accuracy_prev14d	drop_pct	days_since_last_activity	reason
15373	MED	0		5	0.00	1.00	25	Activity dropped >= 60% vs previous 14d; Very low attempts in last 14d;
15372	MED	0		5	0.20	1.00	26	Activity dropped >= 60% vs previous 14d; Very low attempts in last 14d;
15371	MED	0		5	0.80	1.00	26	Activity dropped >= 60% vs previous 14d; Very low attempts in last 14d;
15374	MED	0		5	0.00	1.00	15	Activity dropped >= 60% vs previous 14d; Very low attempts in last 14d;

File CSV (HIGH+MED) terlampir untuk diunduh.

Terima kasih.

Salam,
Kevin

Satu lampiran • Dipindai oleh Gmail ⓘ Tambahkan ke Drive

risk_alert_high_m...

7. Catatan Tambahan

Untuk sekarang identitas siswa masih pakai student_key karena data directory (nama siswa, kelas, email wali kelas) belum lengkap atau belum siap join. Kalau tabel directory sudah rapi, student_key bisa di-join supaya laporan lebih manusiawi (nama siswa + kelas).

Threshold risk (mis. drop_pct, minimum attempts) sengaja dibuat jadi parameter supaya gampang di-tuning sesuai pola penggunaan sekolah.